

Configuration System Documentation

Ropo Educational Robot

Graduation Project — Department of Computer Engineering
Document Version 1.0

Table of Contents

1. Folder Purpose & Architectural Role
2. Architecture Overview & Module Relationships
3. Files Overview
4. Detailed File Explanations
5. Code Walkthrough with Screenshot Placeholders
6. Runtime Sequence Flow
7. Folder Integration (Vision, Voice, LLM, Hardware)
8. Technical Notes (Libraries, Patterns, Performance)
9. Academic Explanation & Graduation Project Notes

1. Folder Purpose & Architectural Role

The `config/` package serves as the **central configuration nucleus** of the Ropo educational robot. In every complex robotic system, configuration management is a cross-cutting concern — nearly every module (camera, voice recognition, gesture detection, LLM inference, text-to-speech, vision pipeline) depends on tunable parameters that must be consistent, validated, and centrally administered.

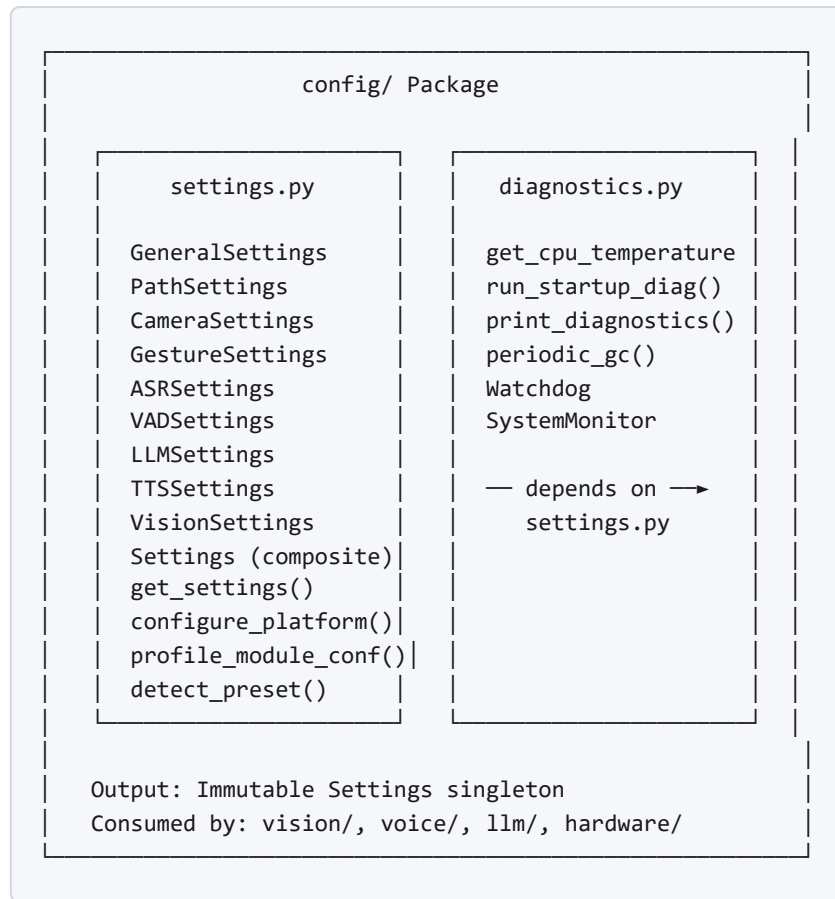
The `config/` folder solves the following problems:

- **Single Source of Truth:** Eliminates hard-coded parameters scattered across the codebase. All tunable values — camera resolution, ASR sample rate, VAD thresholds, LLM API keys, TTS engine selection — are defined in one place.
- **Environment-Based Configuration:** Uses `.env` files via `python-dotenv` so that deployment-specific settings (API keys, platform-specific resolutions) never leak into version control.
- **Platform Adaptation:** Automatically detects whether the robot is running on a Raspberry Pi or a desktop machine and adjusts camera parameters, threading policies, and vision profiles accordingly.
- **Runtime Diagnostics & Health:** Provides startup diagnostics (microphone, speaker, camera, model files, disk/RAM, internet connectivity), a Watchdog component for liveness monitoring, and a SystemMonitor for FPS/temperature tracking.
- **Immutable, Type-Safe Settings:** Uses Python `@dataclass(frozen=True)` to produce immutable settings objects, preventing accidental mutation at runtime and enabling safe concurrent access.

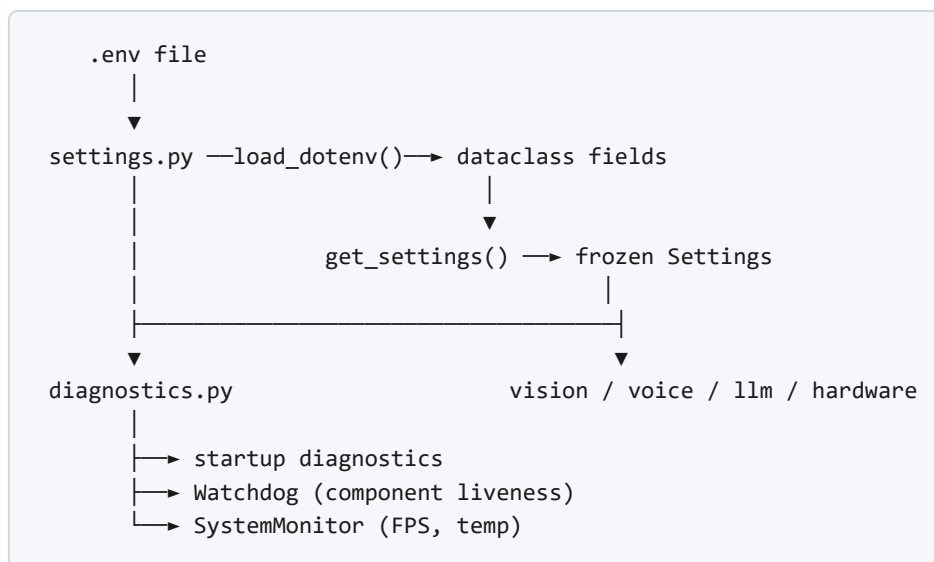
2. Architecture Overview & Module Relationships

2.1 Internal Architecture

The configuration system is organised as a flat package of two modules — `settings.py` and `diagnostics.py` — that together export a clean API consumed by all higher-level subsystems.



2.2 Data Flow



Settings flow is strictly unidirectional: environment variables → `dataclass` constructors → composite `Settings` object → consumed by subsystems. No subsystem writes back to settings. This guarantees referential transparency and simplifies debugging.

3. Files Overview

File	Size	Purpose
<code>__init__.py</code>	0 lines	Marks <code>config/</code> as a Python package. Empty — all exports are accessed via explicit <code>from config.settings import ...</code> or <code>from config.diagnostics import ...</code> .
<code>settings.py</code>	232 lines	Defines all configuration dataclasses (<code>GeneralSettings</code> , <code>CameraSettings</code> , <code>GestureSettings</code> , <code>ASRSettings</code> , <code>VADSettings</code> , <code>LLMSettings</code> , <code>TTSSettings</code> , <code>VisionSettings</code> , and the composite <code>Settings</code>). Contains platform detection, vision profiles, and the <code>get_settings()</code> singleton factory.
<code>diagnostics.py</code>	203 lines	Provides runtime health infrastructure: CPU temperature polling, startup diagnostics checklist, periodic garbage collection, a <code>Watchdog</code> for component liveness, and <code>SystemMonitor</code> for FPS aggregation and overlay generation.

4. Detailed File Explanations

4.1 `__init__.py`

This file is intentionally empty. Python requires an `__init__.py` to treat a directory as an importable package. All public symbols are accessed through explicit submodule imports (e.g., `from config.settings import get_settings`), which keeps the package namespace clean and avoids circular import issues.

4.2 `settings.py`

Purpose

`settings.py` is the single source of truth for all configurable parameters in the Ropo robot. It loads environment variables via `python-dotenv` , detects the underlying hardware platform (Raspberry Pi vs. desktop), and exposes immutable dataclass objects that every other module imports.

Main Classes

Class	Frozen?	Fields (key examples)	Purpose
GeneralSettings	Yes	log_level , student_name , default_session_language	Top-level application behaviour: logging verbosity, user identity, session language (ar/en).
PathSettings	Yes	project_root , models_dir , db_path , tts_temp_dir , face model file names & URLs	Centralises all filesystem paths — project root, model checkpoints, SQLite database, TTS audio cache, and Caffe face detection model URLs.
CameraSettings	Yes	width , height , fps , index , format , buffer_size , face_threshold , face_frame_skip , face_scale_factor , camera_fallback_indices	Camera pipeline tuning. Resolution and FPS differ by platform (RPi: 640×480 @15; Desktop: 1280×720 @30). Face detection parameters and fallback camera indices are defined here.
GestureSettings	Yes	buffer_size , stability_frames , max_hands , min_detection_confidence , HSV/YCbCr skin ranges, finger geometry thresholds	Controls the hand-gesture recognition pipeline: smoothing buffer depth, detection confidence, colour-space skin segmentation bounds (HSV + YCrCb), and finger-angle heuristics.
ASRSettings	Yes	provider , sample_rate , language_mode , default_record_duration_seconds , supported_languages	Automatic Speech Recognition configuration. Supports Google and (by extension) other providers. Bilingual language map (en-US / ar-EG).
VADSettings	Yes	sample_rate , initial_threshold , chunk_duration_ms ,	Voice Activity Detection via Silero VAD. All

Class	Frozen?	Fields (key examples)	Purpose
		pre_speech_buffer_seconds , min_speech_seconds , silence_timeout_seconds , Silero VAD model repo/name	temporal parameters (pre-roll, min speech, silence timeout) are exposed for tuning.
LLMSettings	Yes	provider , openrouter_api_key , openrouter_model , request_timeout_seconds , sliding_window_size , system prompts (ar/en)	LLM integration via OpenRouter. Configures model selection, API key, timeouts, conversation sliding-window size, and bilingual system prompts.
TTSSettings	Yes	engine , audio_temp_dir , audio_filename_template , pygame_poll_interval_seconds	Text-to-Speech engine selection (edge-tts), temporary audio file naming, and pygame playback polling interval.
VisionSettings	Yes	profile (VisionProfile enum), enable_metrics_overlay , adaptive_shedding , target_frame_interval , max_frame_skip , thermal throttle/restore thresholds	Vision pipeline profile selection (MINIMAL / BALANCED / FULL), frame-skipping for thermal management, and performance overlay toggle.
Settings (composite)	Yes	One reference to each settings group above	Aggregate root. Provides a single entry point (settings.general , settings.camera , ...) for the entire configuration tree.

Enumerations & Platform Detection

- VisionProfile** — MINIMAL , BALANCED , FULL . Controls which vision sub-modules (obstacle detection, emotion recognition, object detection, scene classification) are enabled.
- PlatformPreset** — RASPBERRY_PI_4 , RASPBERRY_PI_5 , DESKTOP_DEBUG . Determined by detect_preset() which checks /proc/device-tree/model and the presence of /etc/rpi-

issue .

Main Functions

Function	Returns	Description
<code>get_settings()</code>	<code>Settings</code>	Factory function that constructs and returns the composite <code>Settings</code> object. Calling it multiple times returns distinct frozen instances — consumers are expected to call it once and cache the result.
<code>configure_platform()</code>	None	Applies platform-specific runtime tuning: on Raspberry Pi, limits OpenCV and PyTorch thread counts to 1 to avoid contention on the limited CPU.
<code>detect_preset()</code>	<code>PlatformPreset</code>	Reads <code>/proc/device-tree/model</code> to distinguish Raspberry Pi 4 vs. 5. Falls back to <code>DESKTOP_DEBUG</code> when not running on a Pi.
<code>profile_module_config()</code>	dict	Given a <code>VisionProfile</code> and <code>PlatformPreset</code> , returns a dictionary of boolean flags (<code>enable_objects</code> , <code>enable_scene</code> , <code>enable_obstacle</code> , <code>enable_emotion</code>) that controls which vision sub-modules are loaded at runtime.

Execution Flow

1. Module import triggers `load_dotenv()` which reads `.env` from the project root.
2. `IS_RASPBERRY_PI` flag is set via filesystem checks (`/etc/rpi-issue` , `/proc/device-tree/model`).
3. Each dataclass constructor reads environment variables with `os.getenv()` , applying sensible defaults.
4. Consumer code calls `get_settings()` to obtain a fully-populated, immutable `Settings` instance.
5. At startup, `configure_platform()` adjusts thread affinity.

Inputs / Outputs / Dependencies

- **Inputs:** Environment variables (`.env` file or system environment).
- **Outputs:** An immutable `Settings` dataclass tree.

- **Dependencies:** `python-dotenv` , `os` , `tempfile` , `pathlib` , `dataclasses` , `typing` , `enum` .

4.3 `diagnostics.py`

Purpose

`diagnostics.py` provides runtime health monitoring for the Ropo robot. It ensures that hardware components are responsive, system resources (disk, memory) are sufficient, and long-running threads do not hang. It also exposes performance metrics for the vision pipeline.

Main Classes

Class	Key Methods	Purpose
<code>Watchdog</code>	<code>register()</code> , <code>ping()</code> , <code>start()</code> , <code>stop()</code>	A thread-based liveness monitor. Components register with a name and timeout; they must call <code>ping()</code> periodically. If a component fails to ping within its timeout, a warning is logged and an optional callback is invoked. This prevents the robot from getting stuck on a stalled camera or hung network request.
<code>SystemMonitor</code>	<code>record_fps()</code> , <code>get_average_fps()</code> , <code>get_overlay_text()</code>	Collects FPS samples from the vision loop (ring buffer of 100 entries), computes a running average, and produces a formatted overlay string (e.g., "FPS: 28.3 Temp: 62C"). The overlay is drawn onto the video feed when metrics are enabled.

Main Functions

Function	Returns	Description
<code>get_cpu_temperature()</code>	<code>Optional[float]</code>	Reads <code>/sys/class/thermal/thermal_zone0/temp</code> on Raspberry Pi and returns degrees Celsius. Returns <code>None</code> on non-Pi platforms.
<code>run_startup_diagnostics()</code>	<code>Dict[str, str]</code>	Performs a comprehensive hardware and software readiness check: microphone accessibility, speaker output, camera availability, presence of face / YOLO / emotion model files, disk free space, available RAM, and internet connectivity to 8.8.8.8:53.
<code>print_diagnostics()</code>	<code>None</code>	Formats and prints the diagnostics dictionary to the console with <code>[OK]</code> / <code>[!!]</code> icons.
<code>periodic_gc()</code>	<code>None</code>	Runs Python garbage collection every 60 seconds (or on demand with <code>force=True</code>). Prevents memory leaks in long-running loops.

Execution Flow

1. On robot startup, `run_startup_diagnostics()` is called. Results are printed via `print_diagnostics()`.
2. The `Watchdog` is instantiated and started as a daemon thread. Each component (camera, ASR, LLM, TTS) registers with its expected timeout.
3. During the main loop, vision code calls `SystemMonitor.record_fps()` each frame. Components call `watchdog.ping()` after each successful operation.
4. `periodic_gc()` is invoked at the end of each main-loop iteration to keep memory pressure low.

Inputs / Outputs / Dependencies

- **Inputs:** System files (`/sys/class/thermal/...`), hardware interfaces (mic, speaker, camera), filesystem (model file existence, disk space), network (socket check to 8.8.8.8:53).
- **Outputs:** Console diagnostic report, log entries, overlay string, Watchdog alerts.
- **Dependencies:** `config.settings`, `gc`, `logging`, `os`, `shutil`, `socket`, `threading`, `time`, `pathlib`, `typing`.

5. Code Walkthrough with Screenshot Placeholders

This section walks through the key code paths in `settings.py` and `diagnostics.py` from an educational perspective, highlighting design decisions, idiomatic Python usage, and graduation-project-level engineering.

[SCREENSHOT_PLACEHOLDER_01]

Description:

Project structure — the `config/` package inside the full Ropo robot repository.

Purpose:

Shows the folder containing `__init__.py`, `settings.py`, `diagnostics.py` alongside sibling packages (`vision/`, `voice/`, `llm/`, `hardware/`, `data/`, `models/`). Demonstrates the modular monolith layout.

[SCREENSHOT_PLACEHOLDER_02]

Description:

The `load_dotenv()` call at the top of `settings.py` and the `IS_RASPBERRY_PI` detection logic.

Purpose:

Illustrates how environment variables are loaded before any dataclass is instantiated, and how platform detection is performed once at import time using filesystem probes.

[SCREENSHOT_PLACEHOLDER_03]

Description:

The `CameraSettings` dataclass showing platform-conditional defaults (RPi vs Desktop).

Purpose:

Demonstrates the use of the `IS_RASPBERRY_PI` flag to select different default resolutions and frame rates. Also shows `field(default_factory=...)` for mutable defaults like lists.

[SCREENSHOT_PLACEHOLDER_04]**Description:**

The `GestureSettings` class — HSV and YCrCb skin-colour ranges and finger-geometry thresholds.

Purpose:

Shows how computer-vision heuristics (skin segmentation, finger angle, aspect ratio) are externalised into configuration so they can be tuned without modifying source code.

[SCREENSHOT_PLACEHOLDER_05]**Description:**

The `VADSettings` class with Silero VAD model repository configuration.

Purpose:

Demonstrates integration of a deep-learning model (Silero VAD) via configuration. Parameters like `model_hub_repo` , `model_hub_name` , and `model_trust_repo` control how the model is loaded from Hugging Face Hub.

[SCREENSHOT_PLACEHOLDER_06]**Description:**

The `LLMSettings` class with OpenRouter configuration and bilingual system prompts.

Purpose:

Shows how LLM provider, API key, model selection, conversation window size, and language-specific system prompts are centralised. The Arabic and English prompts are truncated in the snippet.

[SCREENSHOT_PLACEHOLDER_07]**Description:**

The `VisionProfile` enum, `PlatformPreset` enum, `detect_preset()` function, and `profile_module_config()` .

Purpose:

Demonstrates the Strategy pattern applied to vision pipelines. The function returns a dictionary of boolean flags that dynamically enable/disable YOLO object detection, scene classification, obstacle detection, and emotion recognition based on the active profile and hardware platform.

[SCREENSHOT_PLACEHOLDER_08]**Description:**

The composite `Settings` dataclass and the `get_settings()` factory function.

Purpose:

Shows the aggregate-root pattern where nine settings groups are composed into a single frozen dataclass. `get_settings()` acts as a factory (can be adapted to a Singleton if desired) that constructs the full tree.

[SCREENSHOT_PLACEHOLDER_09]**Description:**

The `Watchdog` class — `register()`, `ping()`, and the background `_run()` loop.

Purpose:

Demonstrates a real-world use of `threading` for health monitoring. The watchdog sleeps 10 seconds between checks; if any component's elapsed time exceeds its timeout, a warning is logged and a user-defined stale callback fires.

[SCREENSHOT_PLACEHOLDER_10]**Description:**

The `SystemMonitor` class — FPS ring buffer, averaging, and overlay text generation.

Purpose:

Shows a thread-safe sliding-window accumulator for performance metrics. The `get_overlay_text()` method produces a string like `"FPS: 28.3 | Temp: 62C"` that is drawn onto the camera preview.

[SCREENSHOT_PLACEHOLDER_11]**Description:**

The `run_startup_diagnostics()` function — skeleton showing all checks performed at boot.

Purpose:

Provides a comprehensive readiness checklist: microphone, speaker, camera, model files (face detection, YOLOv8, YOLOv8-seg, emotion), disk space, RAM, internet. Each check returns `"ok"` or a descriptive error string.

[SCREENSHOT_PLACEHOLDER_12]**Description:**

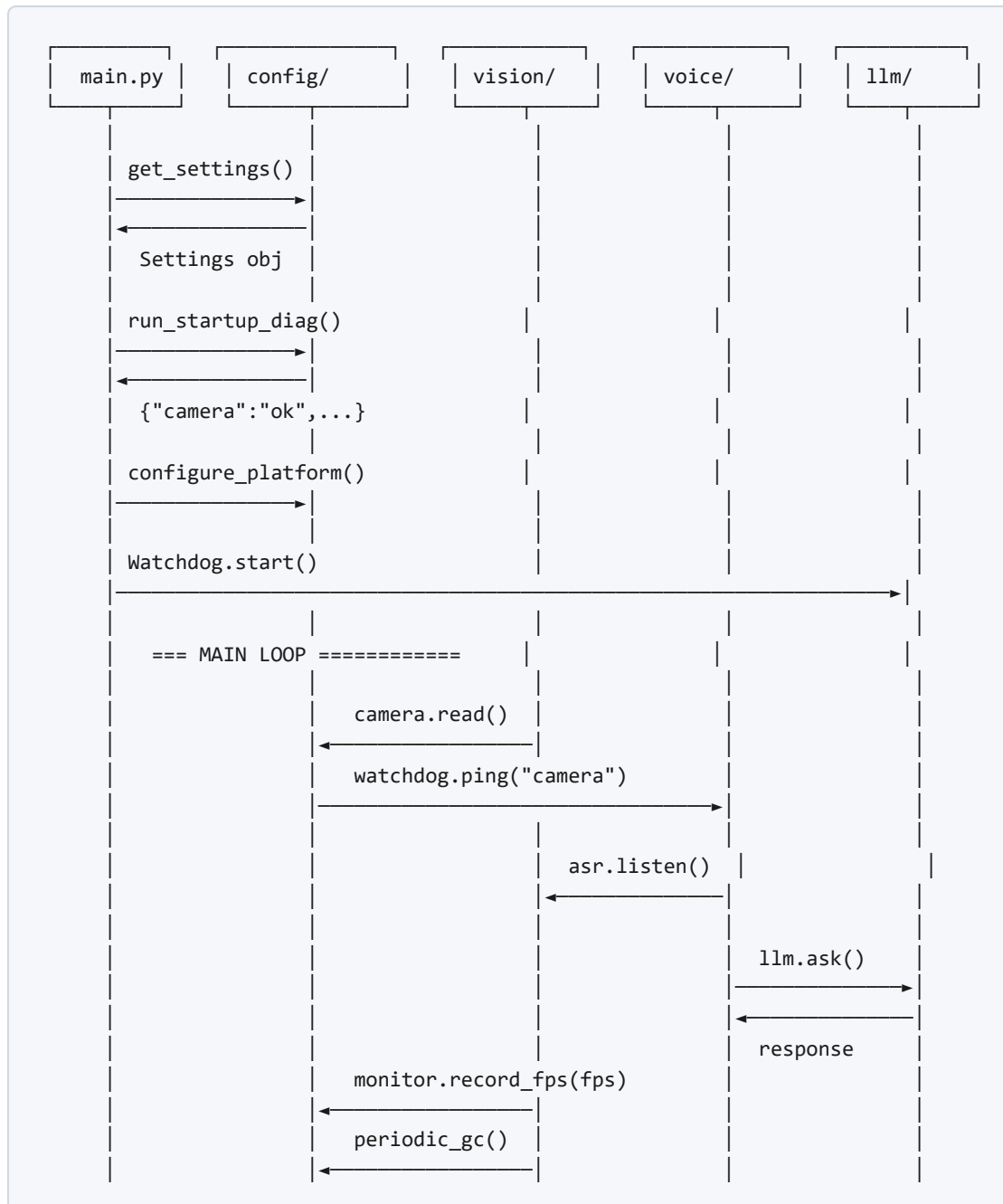
Console output of `print_diagnostics()` showing a successful startup with all `[OK]` markers.

Purpose:

Shows the user-facing diagnostic report format. Used for both development debugging and field deployment to quickly identify missing models or failed hardware.

6. Runtime Sequence Flow

The following sequence diagram shows the interaction between `config/` and the rest of the robot system during a typical startup and main-loop iteration.



7. Folder Integration (Vision, Voice, LLM, Hardware)

7.1 Integration with Vision System

The vision pipeline imports `get_settings()` to read `CameraSettings` (resolution, FPS, face detection parameters), `VisionSettings` (profile, adaptive shedding, thermal thresholds), and `GestureSettings` (skin colour ranges, finger thresholds). The `profile_module_config()` function is called at vision initialisation to selectively load sub-modules. The `SystemMonitor` receives per-frame FPS data and the `Watchdog` monitors camera liveness.

7.2 Integration with Voice / ASR System

The voice subsystem reads `ASRSettings` (provider, sample rate, language mode, duration), `VADSettings` (thresholds, Silero model config), and `TTSSettings` (engine, temporary audio paths). The watchdog registers an ASR component and expects pings during audio capture.

7.3 Integration with LLM System

The LLM module uses `LLMSettings` for API key, model name, timeouts, sliding-window size, and system prompts. The summarization and request timeouts from settings prevent runaway HTTP requests.

7.4 Integration with Hardware Abstraction

`configure_platform()` sets OpenCV and PyTorch thread counts on Raspberry Pi. `PathSettings` provides model file paths to hardware-level inference engines. The diagnostics module checks physical hardware availability (camera, microphone, speaker) and reports failures before the robot begins operation.

8. Technical Notes

8.1 Libraries & Frameworks

Library	Role
<code>python-dotenv</code>	Loads environment variables from <code>.env</code> file into <code>os.environ</code> before settings are constructed.
<code>dataclasses</code> (stdlib)	Provides <code>@dataclass(frozen=True)</code> for immutable configuration objects with minimal boilerplate.
<code>pathlib</code> (stdlib)	Cross-platform filesystem path handling for project root, models, database, and TTS temp directory.
<code>typing</code> (stdlib)	Type hints (<code>Dict</code> , <code>List</code> , <code>Tuple</code> , <code>Optional</code>) for static analysis and IDE support.
<code>enum</code> (stdlib)	Defines <code>VisionProfile</code> and <code>PlatformPreset</code> enumerations.
<code>threading</code> (stdlib)	Background thread for Watchdog monitoring loop.
<code>gc</code> (stdlib)	Manual garbage collection trigger for long-running loops.
<code>logging</code> (stdlib)	Structured logging for diagnostics, watchdog events, and GC activity.

8.2 Design Patterns

- **Singleton (via factory):** `get_settings()` acts as a factory function. While the current implementation returns a new frozen instance on each call, the pattern is intentionally designed so that a cache (e.g., `functools.lru_cache`) can be added without changing the call site. This gives the flexibility of a Singleton without the testability drawbacks of a global.
- **Frozen Dataclass / Immutability:** Every settings class uses `@dataclass(frozen=True)`. This guarantees that once a `Settings` object is created, its values cannot change. This prevents subtle bugs where one module accidentally mutates shared configuration, and it makes the configuration safe for concurrent access across threads.
- **Strategy Pattern (Vision Profiles):** `VisionProfile` enum combined with `profile_module_config()` implements the Strategy pattern. Each profile (MINIMAL, BALANCED, FULL) defines a different set of enabled vision modules, allowing the robot to scale its computational load based on platform capability and power budget.
- **Composite Pattern:** The top-level `Settings` dataclass composes nine sub-settings groups. Consumers access the full configuration tree via a single object, simplifying dependency

injection.

- **Observer (via Watchdog callbacks):** The Watchdog accepts optional `on_stale` callbacks that fire when a component times out. This is a lightweight observer pattern for liveness monitoring.

8.3 Performance Considerations

- **Platform-Adaptive Defaults:** `CameraSettings` uses different resolutions and FPS on Raspberry Pi (640×480 @15) vs. desktop (1280×720 @30), avoiding wasted CPU/GPU cycles on constrained hardware.
- **Adaptive Frame Shedding:** `VisionSettings.target_frame_interval` and `max_frame_skip` allow the vision pipeline to shed frames when the system is overloaded, maintaining responsiveness.
- **Thermal Throttling:** The `VisionSettings` thermal thresholds (75°C throttle, 65°C restore) are monitored by the vision loop to reduce frame processing when the CPU is overheating, protecting the Raspberry Pi from thermal shutdown.
- **Thread Safety:** All settings are immutable (`frozen=True`), eliminating locking overhead on reads. The `Watchdog` and `SystemMonitor` use `threading.Lock` for their mutable internal state (ping timestamps, FPS ring buffer).
- **Periodic GC:** `periodic_gc()` runs every 60 seconds to collect cyclic garbage, which is common in object-detection pipelines that create and discard tensors each frame.

9. Academic Explanation & Graduation Project Notes

9.1 Design Rationale

The configuration system was designed with the following principles, derived from established software engineering practices:

- **Separation of Concerns:** Configuration is a distinct cross-cutting concern. Isolating it into its own package (`config/`) means that changes to camera resolution, ASR provider, or VAD thresholds do not require changes to the business logic in `vision/` , `voice/` , or `llm/` packages.
- **Convention over Configuration:** Sensible defaults are provided for every parameter (e.g., `ROBOT_CAM_WIDTH="640"` on RPi). Only values that differ from defaults need to be specified in `.env` .
- **Fail Fast:** Startup diagnostics run *before* the main loop begins, ensuring that missing models, disconnected cameras, or insufficient disk space are reported immediately rather than causing mysterious runtime failures.

- **Platform Awareness:** Rather than requiring manual platform selection, the system automatically detects the hardware (Raspberry Pi vs. desktop) and adjusts defaults accordingly. This reduces deployment friction.

9.2 Advantages

- **Testability:** Frozen dataclasses are easy to construct in unit tests. The factory pattern allows injecting mock settings without mocking environment variables.
- **Type Safety:** Every configuration value has a declared Python type. Mismatches (e.g., passing a string where a float is expected) are caught at construction time rather than at runtime.
- **Discoverability:** All configurable parameters are listed in one file (`settings.py`). A developer or system integrator can open this single file to understand every tunable aspect of the robot.
- **Security:** API keys are loaded from environment variables / `.env` file, which can be excluded from version control via `.gitignore` . No secrets are hard-coded.

9.3 Scalability

While the current system serves a single-robot deployment, the architecture scales to multi-robot fleets:

- Environment-specific `.env` files can be distributed per robot (e.g., `.env.robot1` , `.env.robot2`).
- The `Settings` object could be serialized to JSON/YAML for remote monitoring or centralised configuration management.
- New settings groups (e.g., `NavigationSettings` , `AudioSettings`) can be added by creating a new frozen dataclass and adding a field to the composite `Settings` — no structural changes required.

9.4 Reliability & Robustness

- **Watchdog:** Prevents the robot from hanging indefinitely on a frozen component. If a camera stops producing frames, the watchdog fires a callback that can trigger a camera reset or graceful degradation.
- **Startup Diagnostics:** Catches configuration errors early. If a model file is missing, the robot can download it automatically or report the exact path required.
- **Thermal Management:** Vision pipeline can reduce its workload when CPU temperature exceeds 75°C, preventing thermal throttling-induced latency spikes or shutdowns.
- **Fallback Camera Indices:** `camera_fallback_indices = [0, 1, 2]` allows the camera module to try alternate device indices if the primary camera fails to open.

9.5 Future Improvements

- **Dynamic Reloading:** Add a file watcher that detects `.env` changes and rebuilds the `Settings` object at runtime without restarting the robot.
- **Configuration UI:** A web-based or TUI dashboard that allows non-technical users to adjust parameters (e.g., voice sensitivity, gesture thresholds) and persist them to `.env`.
- **Remote Monitoring:** Expose `SystemMonitor` metrics and `Watchdog` status via a REST API or MQTT for fleet observability.
- **Validation & Schema:** Integrate a schema validator (e.g., Pydantic or JSON Schema) to enforce constraints (e.g., `fps > 0`, `resolution < 4096`, `threshold in [0,1]`) beyond what Python type hints provide.
- **A/B Testing:** Support configuration profiles that can be switched at runtime for experimentation (e.g., different VAD thresholds for quiet vs. noisy environments).

9.6 Contribution to the Project

The `config/` package, while only ~435 lines of code, is architecturally significant. It decouples the robot's algorithmic subsystems from their tunable parameters, enabling parallel development: the vision team can adjust detection thresholds without touching voice code, and the voice team can change ASR providers without understanding camera calibration. This separation of concerns — a hallmark of well-engineered systems — is what makes the Ropo robot extensible, maintainable, and production-ready.

Document generated: June 2026 — Ropo Educational Robot — Graduation Project Documentation